

AVALOKA: EVIDENCE-GATED AGENTIC AUTOML FOR FAST ANALYTICS AND EFFICIENT MODEL TRAINING AND INFERENCE

Anonymous authors

Paper under double-blind review

ABSTRACT

Despite mature databases and cloud warehouses, getting from “raw tables” to reliable insights and model-ready datasets remains slow and failure-prone. The bottleneck is rarely compute; it is the hidden work of making data *interpretable, consistent, and safe*: schema discovery, normalization and join semantics, quality and bias checks, provenance, and production SRE constraints. We present *Avaloka*, an AI-native, evidence-gated agentic workflow that enables fast analysis by letting users point the system at existing data sources and allowing *self-inferential statistical analysis* to (i) summarize distributions and anomalies, (ii) infer candidate semantics and keys, and (iii) propose verifiable transformations and tests.

Avaloka combines LLM reasoning, a coding agent, and statistical context with a validation chain whose core is deliberately not a rulebook. The planning step emits a *machine-checkable acceptance contract* over the result frame; the generated code is checked by schema-grounded static analysis, executed on a bounded sample, and the result then checked against that contract; a final deterministic pass checks the written narrative against the computed evidence. Training is gated separately by pre-fit integrity checks that block rather than warn. Execution is routed by requested fidelity across three substrates — in-process, Daft/Ray, and Ray on Kubernetes — and every agent LLM resolves through one provider chokepoint with fallback. We report what the shipping system measures rather than proposing a study: a deterministic 17/17 regression suite over the ingestion and data-science paths, a routing-stability result in which three inputs move from 11/12, 10/12 and 12/12 misroutes to 0/12 over twelve trials each, and a four-model routing-accuracy comparison at a total spend of \$0.0817 — with what they do not measure stated plainly. Avaloka has *not* been evaluated on DABstep or any external data-agent benchmark; that work is cited for task-design framing only.

1 INTRODUCTION

Natural language interfaces can dramatically lower the barrier to database analytics, ETL, and model development. However, deploying LLM-driven analytics against production data is primarily an *operational robustness* problem rather than a prompt-quality problem: failures arise from human data orchestration errors (missing joins, wrong grain), schema validation gaps, normalization design defects, and change-management mistakes (breaking migrations, drifted columns, incompatible types). In practice, these issues manifest as SRE incidents: runaway queries, degraded plans after statistics changes, inconsistent lineage, and silent data quality regressions.

This paper frames Avaloka as an *engineering study* of how robust an AI-native analytics and AutoML system can become when correctness is made *checkable* rather than judged. Our hypothesis is that **machine-checkable acceptance contracts, deterministic claim verification, and sample-first execution** reduce production failures more effectively than end-to-end “NL-to-code” generation with an LLM reviewer. The argument comes from this system’s own history: before contracts, the blueprint prompt had accumulated roughly twenty-one numbered special-case rules and the validator roughly twenty bespoke AST visitors, each added after a production failure. Rules only cover failures that have already happened; a contract stated by the plan covers the task at hand.

Avaloka follows two engineering constraints. First, an explicit LangGraph state machine with a typed shared state orchestrates the workflow, so routing is inspectable rather than emergent. Second, planning and verification are separate from execution, and execution is *routed* rather than fixed: the same validated code runs in-process for a quick sample, on Daft/Ray for a portfolio of samples, or as a Ray job on Kubernetes for a full pass (Ray, 2025; Daft, 2025). No JVM is involved.

Contributions.

1. **Contract-gated agentic workflow.** A Planner–Coder–Validator–Executor loop in which the plan emits a closed-vocabulary contract of postconditions on the result frame, and a deterministic validator enforces it against the *executed* result rather than inferring intent from the code.
2. **Schema-grounded static analysis with repair.** Twenty AST passes over the generated pandas/scikit-learn code, three of them transformers that mechanically repair known-unsafe constructs instead of only rejecting them.
3. **Deterministic claim verification.** A terminal gate on the *answer*, not the code, using no LLM, so it cannot itself hallucinate.
4. **Fidelity-routed execution with one inference chokepoint.** Three execution substrates selected from requested fidelity, a preflight cost estimator, and one factory through which all twelve agent LLMs resolve a provider, with fallback.
5. **AutoML without manual feature engineering as a design principle.** Pre-fit integrity checks block training on leaked, duplicated or identifier-like data rather than annotating it.
6. **Measured reliability results, with their limits stated.** We report the suites the system runs, and identify which robustness metrics remain unmeasured.

2 SYSTEM OVERVIEW

Avaloka is engineered as a distributed, multi-agentic orchestration platform designed to bridge the gap between high-level human intent and low-level data execution. The architecture is rooted in *decoupling reasoning from execution*, ensuring that non-deterministic model outputs are checked against stated criteria before results reach a user or expensive compute is committed. Figure 1 gives the shape; the system described here is Avaloka 1.6.0 on Python 3.11–3.12.

Decision-Making (Reasoning) Layer. Reasoning agents (Conversational, Planner, Summariser, Coder, Validator) synthesize plans, draft code, and cross-check against samples and schemas. Validation is not purely static: candidate code is executed during validation, but only against a bounded preview — one hundred rows by default, configurable — so the blast radius of a wrong plan is bounded rather than zero.

Execution Layer. Once validated (and optionally user-reviewed), execution agents run analytics and model workflows, producing artifacts, metrics and audit logs. Three execution nodes sit as siblings in the graph: in-process local execution, Ray, and a Kubernetes job — local being a routed destination, not a fallback. Ingest is carried by nine format connectors (CSV, Excel, Parquet, Avro, Delta, Iceberg, JSON, XML and a shared base) plus the database path below; schema inference uses PyArrow.

Graph-based transitions and shared state. LangGraph represents each agent as a node in a directed graph with explicit routing. A typed state object carries schemas, samples, contracts, code artifacts and results. Agents declare the state keys they read and write; undeclared writes are dropped, and a crashing agent becomes a structured failure on the state rather than a broken graph. Chat history is not the carrier of context, but neither is it the only alternative: the graph’s first node injects retrieved context from an episodic memory plane — similarity-searched prior episodes, preferences and style hints, behind a circuit breaker with a hard retrieval timeout.

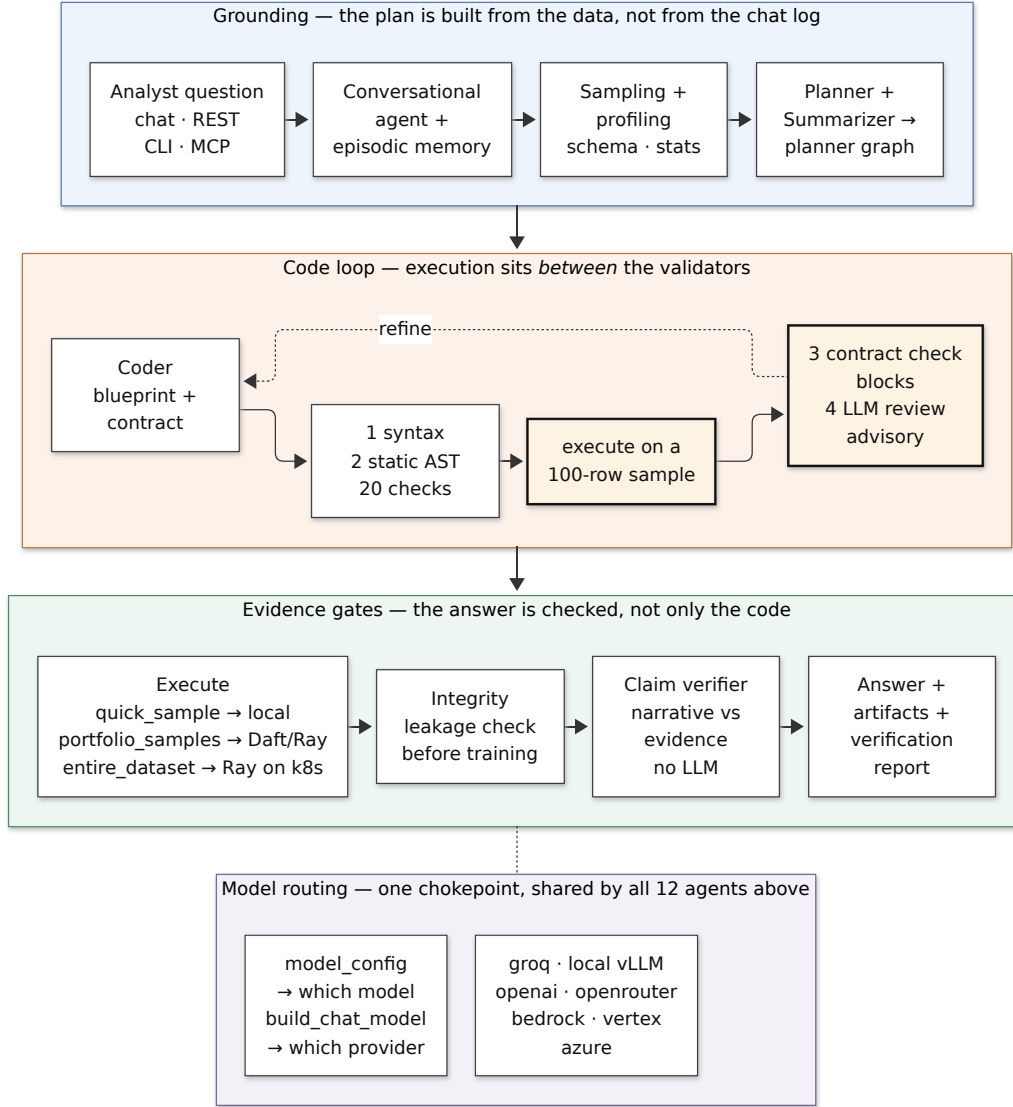


Figure 1: Avaloka in four bands. *Grounding*: the plan is built from sampled, profiled data rather than from the chat log. *Code loop*: execution sits *between* the validators, so the contract check sees a real result frame; the LLM review that follows it is advisory. *Evidence gates*: the answer is checked, not only the code. *Model routing*: one chokepoint shared by all twelve agents. Simplified for print — the full node roster is in Section 5.

Model Context Protocol (MCP), in both directions. Inbound, an MCP server is a standardized connector to enterprise databases, exposing exactly three tools — `query`, `list_tables`, `describe_table` — across PostgreSQL, MySQL, SQLite, SQL Server, Oracle and MongoDB; agents use it for schema discovery and bounded sampling (MCP, 2025). Outbound, Avaloka is an MCP server, exposing mission-level tools (`plan_mission`, `estimate_cost`, `describe_environment`, `inspect_intent`, `visualize`) so an external agent can use Avaloka’s planning and estimation without using its interface.

Core agents. **Planner** parses intent, grounds itself by calling sampling and profiling, and selects the next step. **Coder** produces a blueprint — numbered pseudocode plus a machine-checkable contract — then executable Python grounded in schema previews and samples. **Validator** is four

graph nodes, not one (Section 4). **Executor** dispatches to whichever substrate the routed fidelity selected.

Execution substrate and portability. Avaloka supports distributed execution on Ray and larger-than-memory processing with Daft’s lazy transforms (Ray, 2025; Daft, 2025). Cluster provisioning resolves `local|gcp|aws|azure` through one factory, so analytics execution is portable across clouds and onto a local cluster. We qualify this rather than claiming it wholesale: the model training and serving path still reads GCP project configuration directly, so Avaloka is multi-cloud in the provisioner and single-cloud in serving.

Model routing and fallback. Every agent LLM is built by one factory. A table decides *which model* per agent (twelve agents, each entry environment-overridable); the factory decides *which provider* among seven (Groq, a local OpenAI-spec server such as vLLM or Ollama, OpenAI-spec endpoints, OpenRouter, AWS Bedrock, GCP Vertex, Azure), selectable globally, per role or per agent; and a fallback module decides what happens when a provider fails — a hosted backup, then a local tier sized by deployment profile. Optional provider SDKs load lazily, so a local-only install never needs them. This layer exists because of a real outage: a provider removed an entire model family and six agents failed mid-analysis.

3 WHY “SIMPLE” INSIGHTS AND ML STILL TAKE TOO LONG

A recurring industry paradox is that even a seemingly simple question (e.g., “weekly revenue by segment” or “train a churn model”) can take days to weeks. The delay is not the query itself; it is the prerequisite work to make data *well-defined, stable, and safe* under real-world operational constraints.

Semantic uncertainty and schema reality. Tables rarely encode meaning explicitly: primary keys are implicit, grain is undocumented, business entities are split across systems. Analysts spend time inferring join keys, resolving naming ambiguity, handling nullable identifiers and preventing fanout joins that silently duplicate facts.

Normalization, modeling, and physical-layout defects. Normalization mistakes force repeated ad-hoc fixes downstream — duplicated attributes, snapshots without effective dates, mixed grains in one table, non-idempotent transformations — and usually surface only once dashboards or model scores disagree. Performance compounds this: even read-only analysis depends on partitioning, clustering, indexes and file sizes, and a layout misaligned with access patterns makes “simple” queries expensive and unreliable under concurrency.

Quality, leakage, and change management. Modeling requires more than clean rows: label correctness, missingness mechanisms, spurious correlations, population shift and leakage hazards all have to be identified, which takes iterative profiling and slicing across segments and time windows. Meanwhile the datasets themselves evolve, and schema drift, backfills, late-arriving data and migration errors routinely break jobs or introduce silent regressions, so hardening means contracts, tests, rollbacks and alerting.

SRE constraints and governance overhead. Database SRE concerns (quotas, resource caps, stale statistics, plan regressions) and governance requirements (access control, audit logging, provenance, PII handling) add further steps before data can safely be used.

How Avaloka addresses the bottleneck. Avaloka combines (i) size-adaptive and stratified sampling for fast distributional understanding, (ii) self-inferential profiling to propose constraints, grains and candidate keys, (iii) LLM and coding agents to generate transformations, tests and training code, and (iv) a validation chain that states acceptance criteria up front, executes on a bounded sample, and checks both the result and the narrative against them before a full pass is committed. It addresses semantic uncertainty, fanout and grain defects, leakage, and governance overhead directly; physical layout tuning and schema-migration planning it does not attempt, and Section 8 says so.

4 EVIDENCE-GATED ROBUSTNESS: FROM NL INTENT TO CHECKED RESULTS

Why check the result rather than judge the code. A plan can be syntactically valid, pass a reviewer’s reading, and still return raw rows where an aggregation was asked for, a duplicated group key, or a rate outside $[0, 1]$. Two kinds of check catch that: a rulebook, or a contract stated by the plan for this specific task. Avaloka makes the second its load-bearing gate and keeps the rulebook as a cheap first filter.

The validation chain, in execution order. The coding subgraph wires six nodes: **coder** → **syntax** → **static-semantic** → **execute on sample** → **contract** → **logical-semantic**, with a conditional edge back to the coder. Two of the four checks therefore run *after* a bounded execution, by design: a verdict about a real result frame is stronger than a prediction about one. The static layer derives its known columns from the attached schema and comprises twenty AST passes over pandas and scikit-learn usage — unsafe joins, numeric-versus-string comparisons, dense similarity matrices, oversized grid searches, label-encoder misuse, training on a filtered frame, and others. Three of the twenty are transformers rather than visitors: they repair the code (numeric comparison coercion, whitespace-tolerant string equality, deprecated-API replacement) and emit rewritten source, so a recoverable defect costs no refinement round-trip. The last node is an LLM reviewer, and in the default configuration it *cannot block*: any verdict short of confirmed fabrication is downgraded to an advisory note. It is commentary, not a gate.

Blueprint acceptance contracts. The blueprint step emits, alongside numbered pseudocode, a contract drawn from a small closed vocabulary of postconditions on the result frame. The contract validator checks the executed result against it, and is silent by design when there is nothing trustworthy to check — no contract, no successful execution, or an empty result all leave the pipeline as it was. Because the criteria come from the plan rather than a general rule set, there is nothing for a prompt to memorise.

Sampling: what it is, and what it is not. Sampling grounds the planner and bounds validation cost; it does not answer queries approximately. Database sampling is a uniform random draw expressed per dialect. File and object-store sampling is richer: display and portfolio sizes are chosen from dataset size in five tiers (all rows below one thousand; then 1k/10k, 2k/20k, 10k/50k, capped at 20k/100k above ten million rows), and the portfolio includes K -capped stratified samples carrying bias-correction weights — group size over sampled size, applied where rare categories are oversampled — plus quantile-stratified samples for numeric columns. A preflight estimator maps declared dataset statistics and constraints to a recommended execution mode before compute is committed, and is the only place a statistical target appears: given a margin of error E it sizes $n \approx (z/2E)^2$ at $z = 1.96$, else 1.5% of rows, floored at 50k. For completeness: Avaloka does *not* implement approximate query processing with error-latency guarantees in the sense of BlinkDB (DBDB, 2025), and reads no optimizer plans, cardinality estimates or cost deltas from the underlying database. The only plan-shaped guard is a cross-join block over dataframe row-count metadata, which refuses outright when those counts are unknown. Optimizer integration is future work, not a property of this system.

Checking the claim, not only the code. A correct computation can still be reported wrongly. A terminal gate before the graph’s exit checks the narrative against the computed evidence for five patterns: causal language asserted from correlation, numbers appearing in prose but nowhere in the results, reliability asserted by a model that did not beat its baseline, unhedged extrapolation, and “significant” where no test was run. It is string-level and LLM-free, which is the point: the floor under an LLM-authored narrative should not itself be an LLM.

5 AGENTIC WORKFLOW

The workflow is a state machine with validation gates. The top-level graph wires seventeen nodes:

- **Context and planning:** memory injection (the entry point); conversational agent, which routes before planning; Planner (requirements, iterative analysis, next action); Planner

Graph (renders the plan as an inspectable DAG); Summariser (freezes the plan into a JSON contract).

- **Authoring and checking:** the coding subgraph (Coder plus the four validator nodes and the sample execution between them); structured-action execution.
- **Execution:** infrastructure provisioning; Scheduler; three sibling execution nodes — local, Ray, Kubernetes.
- **Results:** Visualization; Result Narrator; Claim Verifier (terminal gate before the exit).
- **Optional:** Model Training, and Profiling, which is registered for graph routing but currently invoked in-process from the API layer.

Three capability groups sit beside the graph rather than in it, and we distinguish them because the distinction matters for what is gated. *Sampling* is driven from the ingest and planning paths. *PII detection and de-identification* run inside the training and preparation paths. And the *Data Transfer Agent* mirrors the plan–code–validate–execute loop for cross-format movement, alongside a preparation pipeline that proposes dataset repairs without applying them.

Agents exchange a shared state object carrying dataset identifiers, schema snapshots, sampling strategies, blueprint contracts, code artifacts, training configs, endpoint metadata and audit logs, which is what makes scheduled jobs repeatable and retry cheap. The same graph is reached from a web UI, a REST API, two command-line entry points and the outbound MCP server; no capability is exclusive to a surface.

6 SCHEDULING AND PRODUCTION JOBS

Avaloka’s Scheduler agent is built on Celery with RedBeat over Redis: schedules are cron-style parameters stored as Redis-backed entries with a run-count limit each, and users can request periodic jobs, retrieve status, fetch prior results, or cancel tasks. Scheduled runs reuse the validated workflow by re-serializing the typed state into the execution path, so the same validation gates run on every occurrence rather than only on the interactive first run. Training and asynchronous Ray execution dispatch through the same Celery path.

7 AUTOML TRAINING, EFFICIENT FEATURE MINIMIZATION, AND INFERENCE ENDPOINTS

Avaloka’s AutoML path targets an end-to-end outcome: a deployed inference endpoint backed by tracked training artifacts.

Statistical profiling for feature minimization (design principle). Instead of manual feature engineering loops, profiling signals (missingness, cardinality, skewness, leakage risk, stability under sampling) select a minimal feature set sufficient for a baseline model, and candidate features are generated only where profiling indicates predictive gaps.

Pre-fit integrity gating. Both training paths build an integrity report before fitting and *raise* rather than *warn*: duplicate rows shared across splits, identifier-like features (above 0.95 distinct values), and target leakage by correlation above 0.98 each abort the run, and the report — carrying an explicit `safe_to_train` verdict — is recorded with every run. A broader six-check agent adds temporal leakage, near-constant features (below 0.01 distinct) and the target appearing among the features, with graded severities; it is implemented and unit-tested but not yet invoked from the production training path, and we count it as available rather than enforced.

Training workflow. The AutoML Trainer agent (i) produces a training contract (target, splits, metrics, constraints); (ii) generates training code with tests; (iii) runs small-sample pilots to validate data and label integrity; (iv) scales training on Ray; and (v) registers a model and deploys an inference endpoint. Tracking and artifact management run on MLflow.

Evaluation discipline, and one leakage class integrity cannot see. Preprocessing is fitted on the training split only — imputation values, one-hot vocabularies, scaler statistics — and merely applied to validation. Both training paths implement this independently, and no dataframe inspection can catch the leakage class it prevents: the rows are byte-identical either way, and what leaks is a statistic computed at the wrong moment, so unit tests on both paths are the only defence. Above that sits an evaluation module that cross-validates with a splitter chosen from the declared shape of the data — group-aware folds for grouped data, forward chaining for time series, with the choice recorded so it is auditable — and makes a trivial baseline mandatory so every result carries whether it beat that baseline. Like the six-check integrity agent, it is implemented and unit-tested but not yet wired into the production path; we report it as a designed property of the evidence layer, not as a gate that currently fires.

Serving economics. Inference serving runs in one shared cluster, scales to zero when idle, and places workers on preemptible capacity, on the rule that an idle endpoint is pure burn, so idleness must be a state the system reaches on its own rather than a cleanup someone remembers.

8 OPERATIONAL ROBUSTNESS AND FAILURE MODES

We scope robustness to dominant real-world failure modes, and distinguish what the system gates from what it only surfaces.

Human data orchestration errors. Frequent failures include wrong join keys, wrong grain, silent duplication and incorrect assumptions about nullable keys. Avaloka mitigates these with explicit job contracts (inputs, join predicates, expected result shape), static join-safety analysis, and execution on a bounded sample before any full pass. Cross joins are blocked outright when row counts are unknown: an unestimable fanout is not a risk worth taking.

Schema validation. The static validator derives known columns from the attached schema and rejects references outside it, so a renamed or removed column fails at validation rather than at run-time. We claim no drift monitor: nothing compares schemas across time, and a scheduled run inherits whatever schema its fresh ingest attaches, so drift surfaces as a validation failure on the next run — earlier than a corrupted metric, later than an alert.

Normalization and design defects. Avaloka flags likely fanout joins and repeated facts from schema relationships and row-count metadata, and the result contract catches the downstream symptom — a duplicated group key, an inflated row count — even when the cause is not recognised.

Change management errors. This is the clearest gap in the present system: Avaloka does not generate migration plans, does not propose backward-compatible operation sequences, and does not produce rollback instructions. What it provides is lineage — a store of nodes and edges recording ancestry across derivations, which locates PII through the graph rather than by re-scanning — and the audit trail of a scheduled run’s state. Migration planning is future work.

Database SRE difficulties. Avaloka addresses the resource dimension — sample-first execution contains blast radius and the preflight estimator recommends a mode before compute is committed — but does not detect plan regressions or stale statistics, because it reads no optimizer artifacts.

Capability honesty, portability, and offline operation. Optional capabilities resolve by import probe rather than configuration flag, so no environment variable can claim a capability whose code is absent from the build — the property the edition split relies on. Provisioning spans four targets through one factory, keeping governance controls consistent on-prem and across clouds, while serving remains GCP-bound (Section 2). The embedding model is baked into the image and offline embeddings default on, so with a local model server the full stack runs with no network access at all.

What was run	Scale	Result
Deterministic data/analysis suite	17 tasks	17/17, overall 1.00
Conversational suite	77 tests	77 passed (11 live trials, 132 calls)
Agent-contract suite	132 tests	0 failed / 132 passed (from 8 / 121)
Numbered-pick misroute rate	12 trials/input	run 3 11/12 → 0/12 run 5 10/12 → 0/12 run 1 12/12 → 0/12
Planner routing accuracy	54 decisions/model	54/54, 54/54, 54/54, 48/54
Spreadsheet ingest (9 sheets)	1 workbook	datasets 1 → 18; Unnamed 16/16 → 0
Master test plan, live cluster	126 cases	harness 16 (mostly pass), lineage 2/2
Memory recall across a restart	2 calls	3.4 s cold with hints present, 0.6 s warm

Table 1: Measurements on Avaloka 1.6.0. Before/after pairs isolate a single change. The four routing models are an open-weight 120B production default, a frontier model, a 31B instruction-tuned model and a cheap control, costing \$0.0043 / \$0.0743 / \$0.0013 / \$0.0018 — \$0.0817 in total.

9 EVALUATION: MEASURED RELIABILITY, AND WHAT IS NOT MEASURED

We evaluate Avaloka on robustness outcomes and operational reliability rather than raw LLM accuracy, and report results produced by running the system. Where a metric has not been measured, we say so rather than describing an intended protocol as if it had been executed.

9.1 MEASURED RESULTS

Table 1 collects the measurements available for this version; trial counts are stated because a single-shot assertion on a stochastic component is not evidence. Two results deserve comment. The routing fix generalises rather than fitting one model — three models of very different sizes and prices score perfectly, the 31B one at roughly a third of the production default’s cost. And the misroute defect survived a suite that already had eighteen tests for the same resolver precisely because it was intermittent: a single-shot assertion would have passed most runs and been quarantined as flaky, so its regression test now asserts a rate over N trials with zero tolerance.

9.2 WHAT THESE MEASUREMENTS DO NOT ESTABLISH

- **The 17/17 suite cannot compare models.** It scores 17/17 with every API key unset — it never calls an LLM. It exercises Avaloka’s own code (ingestion parity across file, SQLite and object storage; split logic; leakage checks), so a model matrix over it would yield identical scores and imply a discrimination it cannot make. It is a regression floor, not a ceiling.
- **No external benchmark result.** Avaloka has not been run on DABstep (Egg et al., 2025; Adyen and Hugging Face, 2025) or any comparable suite. We cite it for task-design framing — multi-step reasoning over real financial analytics tasks, where the best reported agents reach a low fraction of the hard split — and take safety framing from *AgenticData* (Sun et al., 2025) and the Agents in the Wild workshop (Agents in the Wild, 2026).
- **Coverage is uneven.** The harness is well covered; lineage is real but covered by two cases, which is not coverage of a capability; the memory plane demonstrably stores, persists and retrieves, with no evidence that it improves analyses, and we claim nothing either way on that.
- **Cost and latency are not reported.** Internal per-turn cost tables rest on static prompt sizes, typical rather than measured output lengths, and a price snapshot, so we exclude them.

9.3 ROBUSTNESS METRICS AND THEIR STATUS

- **Prevented execution failures / 100 tasks:** fraction of tasks where a gate stops unsafe execution. *Not measured as a rate* — mechanisms are unit-tested, but the aggregate over a task population is the headline this paper would most want.

- **Contract-catch rate:** executed results rejected by the contract that the LLM reviewer passed. *Not measured*; the cleanest test of the central claim, needing only instrumentation.
- **Schema-violation detection** under synthetic renames and type widenings is *partially covered* by unit tests with no systematic injection study; **cost containment** and **AutoML productivity** (time-to-trainable dataset, feature steps eliminated, time-to-endpoint) are *unmeasured*, the first because the estimator’s coefficients are heuristics awaiting calibration, the rest because each needs a human baseline arm we have not run.

Verification above the suites. Alongside the unit and contract suites (273 test modules), an end-to-end prompt suite drives the real API — upload and register, chat, training, inference, transfer, schedule — across four suites of 86, 96, 44 and 19 items, 245 in total; the last is opt-in and cluster-only because its items materialise hundreds of megabytes in process. The prompt suite asks whether every capability works; the benchmark asks whether the answers are right.

10 LIMITATIONS

Avaloka assumes configured infrastructure (Kubernetes, database credentials, a vector backend) for its distributed and memory-backed features, though offline operation with a local model server is supported. Four limitations bear on the claims above. It reads no optimizer artifacts, so plan regressions and stale-statistics incidents lie outside its detection surface. Its LLM semantic reviewer cannot block by default, so the load-bearing gate is the blueprint contract, and a task whose plan states a weak contract is weakly checked. Serving remains GCP-bound while provisioning is multi-cloud. And the metrics in Section 9 are reported as status rather than results: the mechanisms are implemented and unit-tested, but their aggregate prevention rate is unmeasured.

11 CONCLUSION

Avaloka demonstrates how an evidence-gated, sample-first agentic workflow can operationalize conversational analytics and AutoML while keeping correctness checkable rather than judged: a contract emitted by the plan, schema-grounded static analysis that repairs as well as rejects, bounded execution before any full pass, and a deterministic check of the narrative against the evidence — wrapped in one inference chokepoint and fidelity-routed execution that runs the same validated code on a laptop or a cluster. What we can report is that the regression floor holds and that one reliability defect was driven to zero over repeated trials; the prevention rate that would most strongly support the central claim is the next thing to measure.

REFERENCES

- Sun et al. *AgenticData: An Agentic Data Analytics System for Heterogeneous Data*. arXiv:2508.05002, 2025. <https://arxiv.org/abs/2508.05002>.
- Agents in the Wild (ICLR 2026 Workshop). <https://agentwild-workshop.github.io/>.
- Alex Egg et al. *DABstep: Data Agent Benchmark for Multi-step Reasoning*. arXiv:2506.23719, 2025. <https://arxiv.org/abs/2506.23719>.
- Adyen and Hugging Face. *DABstep Leaderboard (Hugging Face Space)*. <https://huggingface.co/spaces/adyen/DABstep>.
- Database of Databases. *BlinkDB*. <https://dbdb.io/db/blinkdb>.
- Ray documentation. <https://docs.ray.io>.
- Daft documentation. <https://www.getdaft.io>.
- Model Context Protocol. *Specification and Server Concepts*. <https://modelcontextprotocol.io/specification/>.